Transfer and Warm-Start Semantics

What moves between Kaggle notebooks, what warm-start actually loads, and what full resume restores

Why `repo source snapshot` and `checkpoint manifest` were blank under warm-start and full resume. Those two items belong to the *transport and discovery layer*, not to the runtime state-loading layer. Warm-start and full resume both operate *after* a checkpoint directory has already been found. They do not themselves load the packaged repo snapshot or the manifest JSON into the model/trainer state. In the comprehensive table below, those rows are kept and their blank cells are intentional for the same reason.

Key point. In this repo, *cross notebook*, *warm-start*, and *full resume* are different layers:

- **Cross notebook** means physical transport through Kaggle dataset attachment.
- **Warm-start** means runtime initialization that loads the base VLM fresh, attaches a fresh LoRA adapter, and copies only adapter weights from the selected checkpoint.
- **Full same-phase resume** means trainer continuation from a checkpoint, which restores optimizer/trainer/controller state in addition to weights.

Caveat. The exact checkpoint-bundle exporter is not versioned in this repo. Table 1 therefore combines auditable runtime behavior in this repo with the minimum cross-notebook payload that the receiving Kaggle notebooks require.

Table 1: Comprehensive checklist: what each mechanism transports, loads, or restores

Item	Cross Notebook	Warm-Start	Full Resume	Typical File(s)	Lives In	Used By	Source	Brief Description
`repo source snapshot`	●			`staged_rl.tar`, `tests.tar`, repo files	`<code-dataset-root>/`	all Kaggle notebooks	`prepare_kaggle_upload.sh`	Packaged repo copy prepared for Kaggle execution. This is code transfer, not learned model state. The packaging script explicitly excludes local outputs and weight files such as `*.safetensors`, `*.pt`, and `*.pth`.
`checkpoint manifest and routing metadata`	●			`checkpoint_bundle_manifest.json`, selected checkpoint relpath, selected checkpoint metrics	`<checkpoint-dataset-root>/`	Notebook 2 and Notebook 3	`kaggle_staged_train_large_continue.ipynb` `kaggle_staged_train_phase_d_from_large_continue.ipynb`	Discovery and routing metadata used to find the attached checkpoint dataset and locate the checkpoint subtree that becomes `WARM_START_PATH`. It supports handoff discovery, not parameter loading.
`selected checkpoint directory`	●	●	●	`checkpoint-119/`, `checkpoint-120/`, local checkpoint dir	checkpoint dataset subtree or local output checkpoint dir	warm-start loader or trainer resume	notebook `WARM_START_PATH` logic, `staged_rl/checkpointing.py`	Directory that acts as the source artifact. Cross notebook, it is transported by dataset attachment. Warm-start and full resume both point to a checkpoint directory as the source checkpoint object.
`base VLM weights`		●	●	`from pretrained base model`	external model source	model loader	`staged_rl/trainer_runtime.py`	Loaded fresh from `base_model_name` when the new run starts. These weights are not physically transferred through the checkpoint dataset.
`tokenizer / processor config`		●	●	`tokenizer.json`, processor config files	base-model load path or checkpoint dir	model/tokenizer setup	`staged_rl/trainer_runtime.py`	Initialized as part of normal model loading for the new run. This is runtime setup, not the main learned RL transfer payload.
`fresh LoRA adapter scaffold`		●	●	`runtime-created adapter`	in-memory runtime state	Unsloth / PEFT setup	`staged_rl/trainer_runtime.py`	Fresh adapter object attached to the newly loaded base model before any checkpoint state is copied in.
`LoRA adapter weights`	●	●	●	`adapter_model.safetensors`	selected checkpoint directory	warm-start loader and checkpoint resume	`staged_rl/trainer_runtime.py`	The actual learned adapter tensors. This is the key model state transferred across notebook boundaries and also restored on full resume.
`adapter config metadata`	●	●	●	`adapter_config.json`	selected checkpoint directory	PEFT / Unsloth adapter load path	PEFT checkpoint format	PEFT/LoRA metadata needed to interpret the adapter checkpoint correctly.
`optimizer state`			●	`optimizer.pt`	checkpoint dir	full same-phase trainer resume	TRL / Transformers checkpointing	Adaptive optimizer buffers and momentum needed for exact continuation of training dynamics.
`scheduler state`			●	`scheduler.pt`	checkpoint dir	full same-phase trainer resume	TRL / Transformers checkpointing	Learning-rate scheduler progress so resume continues on the same LR schedule.
`AMP scaler state`			●	`scaler.pt`	checkpoint dir	full same-phase trainer resume	TRL / Transformers checkpointing	Mixed-precision gradient-scaling state when present.
`RNG state`			●	`rng_state.pth`	checkpoint dir	full same-phase trainer resume	TRL / Transformers checkpointing	Random-number generator snapshots used to continue from approximately the same stochastic training state.
`trainer progress / log history`			●	`trainer_state.json`, `training_args.bin`	checkpoint dir	full same-phase trainer resume	TRL / Transformers checkpointing	Global step, epoch progress, callback state, log history, and trainer bookkeeping.
`reward controller state`			●	`controller_state.json`	checkpoint dir	same-phase reward-controller restore	`staged_rl/trainer_runtime.py`	This repo's staged-RL-specific controller history and active reward-weight state. Warm-start does not carry this across phases.
`checkpoint reward weights snapshot`			●	`reward_weights.json`	checkpoint dir	same-phase reward-controller restore and audit	`README_staged_rl.md`, `staged_rl/trainer_runtime.py`	Saved reward-weight snapshot paired with the controller state for same-phase continuation and auditability.

Legend. ● means that the item is transported, loaded, or restored by that mechanism.

Practical reading.

- **Notebook 1 → Notebook 2** and **Notebook 2 → Notebook 3** are Kaggle dataset handoffs followed by adapter-only warm-start.
- **Large Phase C → Large Phase D in the same notebook** is still warm-start, not full resume, because it is a cross-phase start.

- Only a **same-phase restart** uses full trainer resume semantics.

Source logic
`prepare_kaggle_upload.sh`: code dataset packaging and exclusions
`kaggle_staged_train_large_continue.ipynb`: manifest discovery and `WARM_START_PATH`
`kaggle_staged_train_phase_d_from_large_continue.ipynb`: same continuation pattern for Notebook 3
`staged_rl/checkpointing.py`: warm-start vs trainer-resume selector logic
`staged_rl/trainer_runtime.py`: fresh model load, fresh LoRA attachment, adapter warm-start, and same-phase resume behavior
`README_staged_rl.md`: checkpoint-side artifact inventory